Bài 3. Trao đổi dữ liệu sử dụng giao thức AMQP (RabbitMQ)

Phần 1. Bài lab trên lớp (theo nhóm)

Yêu cầu. Mỗi nhóm xây dựng hệ thống gồm một producer và một consumer trao đổi dữ liệu qua một RabbitMQ broker (Hình 1):

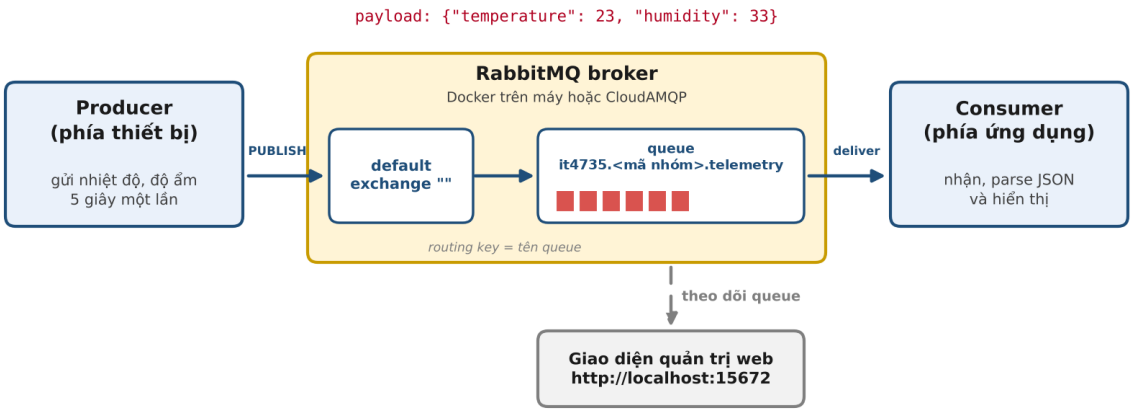
- **Broker**: chạy RabbitMQ bằng Docker trên một máy của nhóm, hoặc tạo một instance CloudAMQP miễn phí (gói **Little Lemur**). Nếu producer và consumer chạy trên hai máy khác nhau thì dùng CloudAMQP — tài khoản `guest` chỉ dùng được từ localhost.
- **Producer (phía thiết bị)**: tạo thông tin cảm biến nhiệt độ, độ ẩm (ví dụ: nhiệt độ 23 °C, độ ẩm 33%), đóng gói dưới dạng JSON và gửi vào queue `it4735.<mã nhóm>.telemetry` qua default exchange, 5 giây một lần.
- **Consumer (phía ứng dụng)**: nhận bản tin từ queue trên, phân tích (parse) dữ liệu JSON nhận được và hiển thị nhiệt độ, độ ẩm.
- Mở giao diện quản trị web của RabbitMQ và theo dõi queue. Dừng consumer khoảng 30 giây: số bản tin **Ready** tăng dần. Chạy lại consumer: consumer nhận hết các bản tin đang chờ — queue giữ lại bản tin, khác với topic của MQTT.

Ví dụ tên queue, nội dung bản tin (payload) và lệnh Docker chạy broker trên máy:

```
Queue:   it4735.nhom01.telemetry
Payload: {"temperature": 23, "humidity": 33}

docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 \
    rabbitmq:4-management
```

Giao diện quản trị web: `http://localhost:15672` (tài khoản guest / guest, chỉ dùng từ localhost).



Hình 1. Mô hình hệ thống của bài lab trên lớp

Yêu cầu nộp bài:

Viết báo cáo mô tả ngắn gọn gồm: mã nguồn chính của chương trình (kèm giải thích), ***chụp ảnh màn hình thực hiện chương trình và kết quả ở các yêu cầu*** (có ảnh chụp queue trên giao diện quản trị web).

Nộp bài: file báo cáo và file mã nguồn (mỗi nhóm cử 1 bạn nộp).

Phần 2. Bài tập về nhà (cá nhân)

Yêu cầu. Viết một cặp chương trình (producer và consumer) bằng ngôn ngữ tùy chọn (C#, Java, Python, …) thực hiện các yêu cầu sau:

a) Gửi dữ liệu lên RabbitMQ broker (producer)

Kết nối tới broker — CloudAMQP qua TLS (cổng 5671), hoặc broker Docker trên máy:

```
amqps://<user>:<password>@<host>/<vhost>   # CloudAMQP (TLS, cổng 5671)
amqp://guest:guest@localhost:5672          # broker Docker trên máy
```

Khai báo queue **bền vững** (durable) `it4735.<MSSV>.telemetry`, đóng gói dữ liệu bằng JSON và gửi dưới dạng bản tin **persistent** với content type `application/json`. Ví dụ dữ liệu:

```
{
    "DeviceId": 11,
    "packet_no": 126,
    "temperature": 30,
    "humidity": 60
}
```

Chương trình gửi dữ liệu định kỳ (ví dụ 5 giây một lần); sau mỗi lần gửi, giá trị packet_no tăng thêm 1.

b) Nhận dữ liệu từ broker (consumer)

Nhận bản tin từ queue trên, sau đó parse dữ liệu nhận được và hiển thị các trường tương ứng: DeviceId, packet_no, temperature, humidity. Khai báo queue với đúng tên và đúng thuộc tính như producer (nếu khác, RabbitMQ báo lỗi PRECONDITION_FAILED).

Chương trình phải bắt lỗi (try/catch) khi nhận bản tin không đúng định dạng JSON: in thông báo lỗi và tiếp tục chạy, không được dừng.

c) Kiểm tra bằng giao diện quản trị web

Trên giao diện quản trị của RabbitMQ (hoặc CloudAMQP): (1) mở queue và quan sát tốc độ bản tin khi chương trình chạy; (2) dừng consumer, gửi vài bản tin, xem số **Ready**, rồi

chạy lại consumer và kiểm tra không thiếu packet_no nào; (3) dùng chức năng **Publish message** trên trang của queue để gửi một bản tin JSON và một bản tin sai định dạng (ví dụ hello), kiểm tra consumer parse đúng bản tin thứ nhất và báo lỗi bản tin thứ hai.

d) Phần mở rộng (khuyến khích)

- Fanout: khai báo fanout exchange `it4735.<MSSV>.broadcast`; chạy hai consumer, mỗi consumer có một queue tạm gắn (bind) vào exchange; cả hai phải nhận được mọi bản tin.
- Topic: khai báo topic exchange `it4735.<MSSV>.topic` và gửi với routing key như `sensor.room1.temperature`, `sensor.room2.humidity`; một consumer bind `sensor.room1.*`, consumer khác bind `sensor.*.temperature`. Giải thích mỗi consumer nhận được những bản tin nào.

Lưu ý: Mật khẩu CloudAMQP cho toàn quyền truy cập broker của bạn. Không ghi mật khẩu trong mã nguồn nộp bài, trên repository công khai hay trong nhóm chat — đọc từ biến môi trường (ví dụ CLOUDAMQP_URL). Dùng useSslProtocol() / URL amqps:// không kiểm tra chứng thư chỉ chấp nhận trong bài lab.

Bước 1: Chuẩn bị broker

1. Cách A – Docker: chạy lệnh ở Phần 1, sau đó mở `http://localhost:15672` và đăng nhập **guest / guest**.
2. Cách B – CloudAMQP: đăng ký tại https://www.cloudamqp.com → **Create New Instance** → gói **Little Lemur (free)**, chọn region gần → **Create instance**.
3. Mở instance → **AMQP details**: lấy user (cũng là vhost), password và URL `amqps://…`. Nút **RabbitMQ Manager** mở giao diện quản trị web.

Bước 2: Chuẩn bị thư viện

Java – thư viện RabbitMQ Java client amqp-client 5.16.0 cùng slf4j-api và slf4j-simple 1.7.36 (tải file .jar tại Maven Central, giống tutorial chính thức):

```
https://repo1.maven.org/maven2/com/rabbitmq/amqp-client/5.16.0/
```

Hoặc khai báo dependency trong file pom.xml nếu dùng Maven:

```
<dependency>
    <groupId>com.rabbitmq</groupId>
    <artifactId>amqp-client</artifactId>
    <version>5.16.0</version>
</dependency>
```

Thư viện JSON cho Java: org.json (https://mvnrepository.com/artifact/org.json/json), giống Bài 2. Thêm file .jar vào project Eclipse: chuột phải vào project → Properties → Java Build Path → tab Libraries → Add External JARs… → chọn file .jar → Apply and Close.

Python – cài thư viện bằng lệnh `pip install pika`; dùng module json có sẵn để parse dữ liệu.

C# – cài gói bằng lệnh `dotnet add package RabbitMQ.Client`; dùng System.Text.Json để parse dữ liệu.

Bước 3: Thiết lập kết nối

Java — đọc URL CloudAMQP từ biến môi trường:

```
ConnectionFactory factory = new ConnectionFactory();
factory.setUri(System.getenv("CLOUDAMQP_URL")); // amqps = TLS
// Local Docker broker: factory.setHost("localhost");
```

Python:

```
import os, pika, json
params = pika.URLParameters(os.getenv("CLOUDAMQP_URL"))
connection = pika.BlockingConnection(params)
```

Mã nguồn tham khảo: https://www.rabbitmq.com/tutorials/tutorial-one-java và https://www.rabbitmq.com/tutorials/tutorial-one-python

Bước 4: Kết quả mong đợi

Ví dụ kết quả chạy chương trình Java mẫu (MSSV 20231234), producer và consumer chạy ở hai cửa sổ terminal. Dòng cuối cho thấy bản tin sai định dạng ("hello", gửi từ giao diện quản trị web) được bắt lỗi mà consumer vẫn chạy tiếp:

Cửa sổ producer:

```
Connected to amqps://<host>/<vhost>
Queue it4735.20231234.telemetry declared (durable)
>> Sent packet_no=126 (62 bytes)
>> Sent packet_no=127 (62 bytes)
...
```

Cửa sổ consumer:

```
 [*] Waiting for messages on it4735.20231234.telemetry. To exit press CTRL+C
<< Received (62 bytes): DeviceId=11, packet_no=126, temperature=30.0, humidity=60.0
<< Received (62 bytes): DeviceId=11, packet_no=127, temperature=30.0, humidity=60.0
...
<< Received (5 bytes): hello
   JSON error: A JSONObject text must begin with '{' at 1 [character 2 line 1]
```

Yêu cầu nộp bài:

Viết báo cáo mô tả ngắn gọn gồm: mã nguồn chính của chương trình (kèm giải thích), ***chụp ảnh màn hình thực hiện chương trình và kết quả ở các yêu cầu***.

Nộp bài: file báo cáo và file mã nguồn (không chứa mật khẩu).